13강. 상속 - II

- protected 접근 제한자
- 타입 변환
- 다형성
- 객체 타입 확인

13-1. protected 접근 제한자

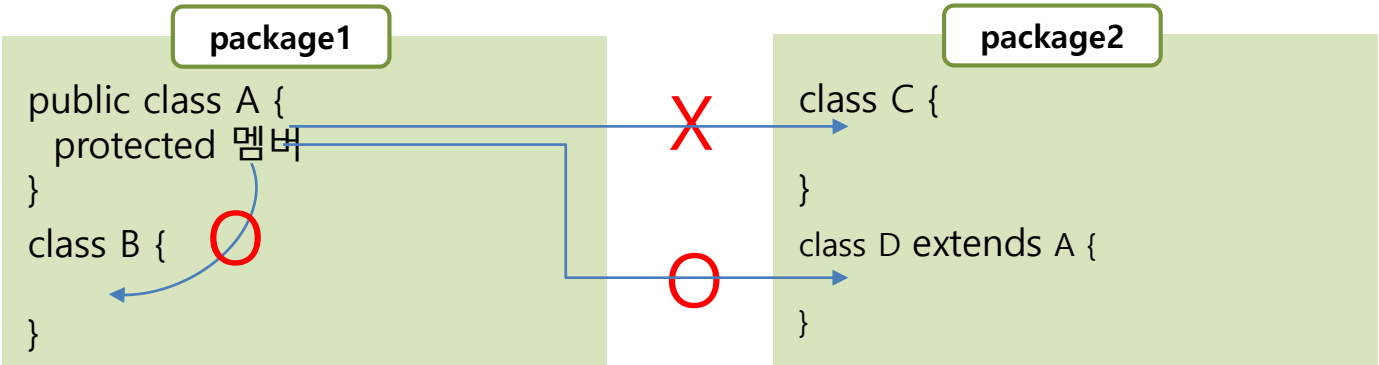
protected 접근 제한자(java_13_1_ex1_inheritance)

➢ 접근 제한자 protected는 상속(Inheritance)과 관련이 있고 public과 default 중간 단계에 해당하는 접근 제한을 한다.

주) default 는 접근 제한자가 아니라 접근 제한자가 붙지 않은 상태를 말한다.

접근 제한자	제한 대상	제한 범위
protected	필드, 생성자, 메서드	같은 패키지이거나, 자식 객체만 사용 가능

➢ protected는 같은 package에서는 default처럼 접근이 가능하고 다른 package에서는 자식 클래스만 접근이 가능하다. protected는 필드, 생성자, 메서드 선언에 사용될 수 있다.



13-2. 타입 변환

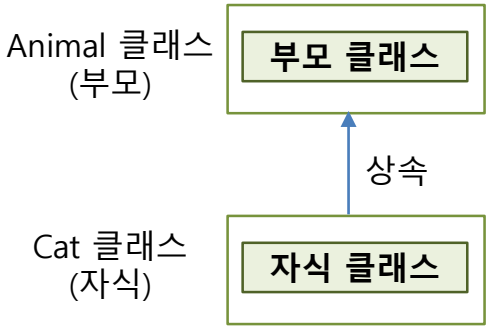
자동 타입 변환(java_13_2_ex1_inheritance)

- 타입 변환이란 타입을 다른 타입으로 변환하는 것을 말한다. 클래스도 타입 변환이 있으며, 클래스의 타입 변환은 상속 관계에 있는 클래스 사이에서 발생한다.
- 자동 타입 변환(promotion)은 자동으로 타입 변환이 일어나는 것을 말하며, 아래의 조건에서 일어난다.

자동타입변환

부모타입 변수 = 자식타입객체;

- 자식은 부모의 특징과 기능을 상속받기 때문에 부모와 동일하게 취급될 수 있다. 아래 예제처럼 고양이가 동물의 특징과 기능을 상속 받았다면, '고양이는 동물이다 ' 가 성립한다.




```
class Animal {
  ...
}
```

상속

```
class Cat extends Animal {
  ...
}
```

- Cat 객체를 생생하고 이것을 Animal 변수에 대입하면 자동 타입 변환이 일어난다.

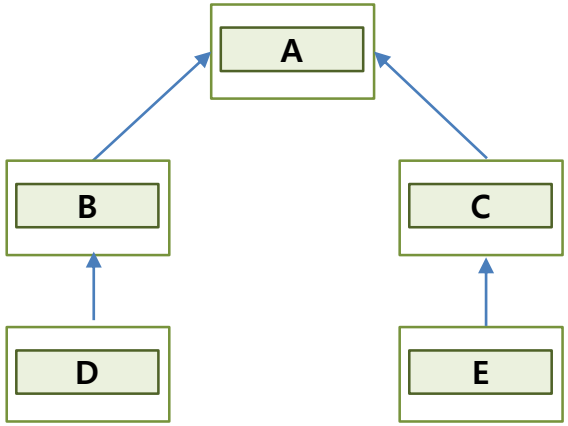
```
Cat cat = new Cat();
Animal animal = cat;
```

Animal animal = new Cat(); 가능

13-2. 타입 변환

자동 타입 변환(java_13_2_ex1_inheritance)

➢ 바로 위의 부모가 아니더라도 상속 계층에서 상위 타입이라면 자동 타입 변환이 일어날 수 있다.




```
B b = new B();
C c = new C();
D d = new D();
E e = new E();
```

```
A a1 = b;     (가능)
A a2 = c;     (가능)
A a3 = d;     (가능)
A a4 = e;     (가능)

B b1 = d;     (가능)
C c1 = e;     (가능)

B b2 = e;     (불가능)
C c2 = d;     (불가능)
```

➢ 부모 타입으로 자동 타입 변환된 이후에는 부모 클래스에 선언된 필드와 메서드만 접근 가능하다. 그러나 자식 클래스에서 오버라이딩된 메서드가 있다면 부모 메서드 대신 오버라이딩된 메서드가 호출된다. 이것은 다형성과 관련이 있습니다.

```
class Parent {
  void method1() { ... }
  void method2() { ... }
}
```

상속

```
class Child extends Parent {
  void method2() { ... }    // 오버라이딩
  void method3() { ... }
}
```

```
class ChildExam {
  public static void main(String[] args) {
  Child child = new Child();
  Parent parent = child;

  parent.method1();                    // 호출 가능
  parent.method2();                    // 호출 가능

  parent.method3();                    // 호출 불가능
}
```

13-2. 타입 변환

강제 타입 변환(java_13_2_ex2_inheritance)

- 자식 타입은 부모 타입으로 자동 변환되지만, 반대로 부모 타입은 자식 타입으로 자동 변환되지 않는다. 강제 타입 변환은 아래와 같이 캐스팅(casting) 연산자로 강제 타입 변환시킨다.

강제타입변환

```
자식타입 변수 = (자식타입) 부모타입객체;
```

캐스팅연산자

- 부모 타입 객체를 자식 타입으로 무조건 강제 변환할 수 있는 것은 아니다. 자식 객체가 부모 타입으로 자동 변환된 후 다시 자식 타입으로 변환할 때 강제 타입 변환을 사용할 수 있다.

```
Parent parent = new Child();          // 자동 타입 변환
Child child = (Child) parent;         // 강제 타입 변환
```

- 자식 객체가 부모 타입으로 자동 변환하면 부모 타입에 선언된 필드와 메서드만 사용 가능하다. 만약 자식 타입에 선언된 필드와 메서드를 꼭 사용해야 한다면 강제 타입 변환해서 자식 타입으로 변환해야 한다.

```
class Parent {
  String filed1;
  void method1() { ... }
  void method2() { ... }
}
```

상속

```
class Child extends Parent {
  String filed2;
  void method3() { ... }
}
```

```
class ChildExam {
  public static void main(String[] args) {
  Parent parent = new Child();
  parent.field1 = "홍길동";              // 사용 가능
  parent.method1();                      // 사용 가능
  parentmethod2();                       // 사용 가능
  parent.filed2 = "홍순이";              // 사용 불가능
  parent.method3();                      // 사용 불가능

  Child child = (Child) parent;
  child.field2();                        // 호출 가능
  child.method3();                       // 호출 가능
} }
```

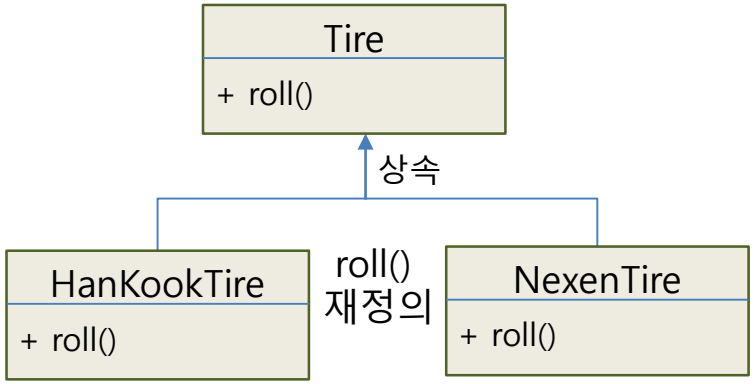
13-3. 다형성

- 다형성이란 사용 방법은 동일하지만 실행 결과가 다양하게 나오는 성질을 말한다. 자동차를 예를 들면 자동차의 부품을 교환하면 성능이 다르게 나오듯이 객체는 부품과 같아서, 프로그램을 구성하는 객체를 바꾸면 프로그램 실행 성능이 다르게 나올 수 있다.
- 다형성을 구현하기 위해서는 자동 타입 변환과 메서드 재정의가 필요하다.

자동 타입 변환 + 메서드 오버라이딩 → 다형성

필드 다형성(java_13_3_ex1_inheritance)

- 필드 다형성은 필드 타입은 동일하지만(사용 방법은 동일하지만), 대입되는 객체가 달라져서 실행 결과가 다양하게 나올 수 있는 것을 말한다.




```
class Car {
  // 필드 선언
  public Tire tire;

  // 메서드 선언
  public void run() {
    tire.roll()          // tire 필드에 대입된 객체의 roll() 메서드 호출
  }
}
```

13-3. 다형성

```
Car myCar = new Car();              // Car  객체 생성

// HankookTire 장착
myCar.tire = new HankookTire();  // 자동 타입 변환

// NexenTire 장착
myCar.tire = new NexenTire();      // 자동 타입 변환
```

- Car 클래스의 run() 메서드는 tire 필드에 대입된 객체의 roll() 메서드를 호출한다. 위의 예제처럼 HankookTire와 NexenTire가 roll() 메서드를 재정의하고 있다면 재정의된 roll() 메서드가 호출된다.

```
myCar.run();       -> 대입된 tire의 roll() 메서드가 호출
```

즉, 장착한 tire에 따라 roll() 메서드의 실행 결과가 달라지게 된다. 이것이 필드의 다형성이다.

13-3. 다형성

- **필드의 다형성 – 사용 예**

```java
public class Tire {
    // 메서드 선언
    public void roll() {
        System.out.println("회전합니다.");
    }
}
public class HankookTire extends Tire {
    // 메서드 재정의(오버라이딩)
    @Override
    public void roll() {
        System.out.println("한국타이어가 회전합니다.");
    }
}

public class Car {
    // 필드 선언
    public Tire tire;

    // 메서드 선언
    public void run() {
        tire.roll();    // tire 필드에 대입된 객체의 roll() 메서드 호출
    }
}
```

```java
public class NexenTire extends Tire {
    // 메서드 재정의(오버라이딩)
    @Override
    public void roll() {
        System.out.println("넥센타이어가 회전합니다.");
    }
}

public static void main(String[] args) {
    // Car 객체 생성
    Car myCar = new Car();

    // Tire 객체 대입
    myCar.tire = new Tire();
    myCar.run();

    // 한국타이어 객체 대입
    myCar.tire = new HankookTire();
    myCar.run();

    // 넥센타이어 객체 대입
    myCar.tire = new NexenTire();
    myCar.run();
}
```

13-3. 다형성

매개변수 다형성(java_13_3_ex2_inheritance)

- 다형성은 필드보다 메서드를 호출할 때 많이 발생한다.메서드가 클래스 타입의 매개 변수를 가지고 있을 경우, 호출할 때 동일한 타입의 객체를 제공하는 것이 정석이지만 자식 객체를 제공할 수도 있다. 여기서 다형성이 발생한다.

다음과 같이 Driver 클래스가 있고 Vehicle 매개 변수를 갖는 driver() 메서드가 정의되어 있다고 가정해보자

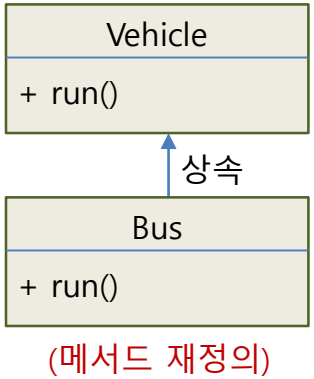
```
public class Driver {
  public void drive(Vehicle vehicle) {
    vehicle.run();
  }
}
```

일반적으로 drive() 메서드를 호출한다면 다음과 같이 Vehicle 객체를 제공할 것이다.

```
Driver driver = new Driver();
Vehicle vehicle = new Vehicle();

driver.drive(vehicle);
```

그러나 자동 타입 변환으로 Vehicle의 자식 객체도 제공할 수 있다.




```
Driver driver = new Driver();
Bus bus = new Bus();

driver.drive(bus);
```

Bus 객체의 run() 호출

자동 타입 변환 발생
Vehicle vehicle = bus;

13-3. 다형성

- drive() 메서드는 매개변수 vehicle이 참조하는 객체의 run() 메서드를 호출하는데 자식 객체가 run() 메서드를 재정의하고 있다면 재정의된 run()메서드가 호출된다.
- 제공되는 자식 객체에 따라 drive() 실행 결과가 달라진다. 이것이 매개변수의 다형성이다.

자식 객체

```
void drive( Vehicle vehicle ) {
  vehicle.run();
}
```

자식 객체가 재정의한 run() 메서드 호출

13-3. 다형성

- **매개변수 다형성 – 사용 예**

```java
public class Vehicle {
    // 메서드 선언
    void run() {
        System.out.println("차량이 달립니다.");
    }
}
public class Bus extends Vehicle {
    // 메서드 재정의(오버라이딩)
    @Override
    void run() {
        System.out.println("버스가 달립니다.");
    }
}
public class Taxi extends Vehicle {
    // 메서드 재정의(오버라이딩)
    @Override
    void run() {
        System.out.println("택시가 달립니다.");
    }
}
```

```java
public class Driver {
    // 메서드 선언(클래스 타입의 매개변수를 가지고 있음)
    public void driver(Vehicle vehicle) {
        vehicle.run();
    }
}

public class DriverEx {
    public static void main(String[] args) {
        Driver driver = new Driver();

        Bus bus = new Bus();
        driver.driver(bus);

        Taxi taxi = new Taxi();
        driver.driver(taxi);
    }
}
```

13-4. 객체 타입 확인

객체 타입 확인(java_13_4_ex1_inheritance)

- 변수가 참조하는 객체의 타입을 확인하고자 할 때 instanceof 연산자를 사용할 수 있다.
- instanceof 연산자의 좌항에 객체가 오고 우항에 타입이 오는데 좌항의 객체가 우항의 타입이면 true, 그렇지 않으면 false 이다.

```
boolean result = 객체 instanceof 타입;
```

- 아래의 예는 Child 타입으로 강제 타입 변환하기 전에 매개 값이 Child 타입인지 여부를 instanceof 연산자로 확인한다.

```
                Parent객체        Child객체

public void method(Parent parent) {
  if( parent instanceof Child ) {
    Child child = (Child) parent;
    // Child 변수 사용
  }
}
```

parent 매개변수가 참조하는 객체가 Child인지 조사

- Java12부터 instanceof 연산의 결과 true 일 때, 우측 타입 변수를 사용할 수 있어 강제타입 변환이 필요 없다.

```
if(parent instanceof Child child) {
    // child 변수 사용
}
```

13-4. 객체 타입 확인

- **객체 타입 확인 – 사용 예**

```java
public class Person {
    public String name;

    public Person(String name) {
        this.name = name;
    }

    public void walk() {
        System.out.println("걸습니다.");
    }
}
public class Student extends Person {
    public int studentNo;

    public Student(String name, int studentNo) {
        super(name);
        this.studentNo = studentNo;
    }

    public void study() {
        System.out.println("공부를 합니다.");
    }
}
```

```java
public class StudentEx {
    public static void personInfo(Person person) {
        System.out.println("name: " + person.name);
        person.walk();

        // Student 타입인 경우 강베타입변환해서 사용
        if(person instanceof Student) {
            Student student = (Student) person;
            System.out.println("studentNo: " + student.studentNo);
            student.study();
        }
        // student 변수에 대입(강테타입변환): ㅓㅁㅡㅁ 12부터 가능
        if(person instanceof Student student) {
            System.out.println("studentNo: " + student.studentNo);
            student.study();
        }
    }

    public static void main(String[] args) {
        Person person1 = new Person("홍길이");
        personInfo(person1);

        System.out.println();

        Person person2 = new Student("홍순이", 30);
        personInfo(person2);
    }
}
```

➢ 예제 1

자바의 상속에 대한 설명 중 틀린 것은 무엇입니까?

① 자바는 다중 상속을 허용한다.
② 부모의 메서드는 자식 클래스에서 재정의(오버라이딩)할 수 있다.
③ 부모의 private 접근 제한을 갖는 필드와 메서드는 상속의 대상이 아니다.
④ final 클래스는 상속할 수 없고, final 메서드는 오버라이딩할 수 없다.

➢ 예제 2

클래스의 타입 변환에 대한 설명 중 틀린 것은 무엇입니까?

① 자식 객체는 부모 타입으로 자동 타입 변환된다.
② 부모 객체는 어떤 자식 타입으로도 강제 타입 변환된다.
③ 자동 타입 변환을 이용해서 필드와 매개변수의 다형성을 구현한다.
④ 강제 타입 변환 전에 instanceof 연산자로 변환 가능한지 검사하는 것이 좋다.

➢ 예제 3

final 키워드에 대한 설명으로 틀린 것은 무엇입니까?

① final 클래스는 부모 클래스로 사용할 수 있다.
② final 필드는 초기화된 후에는 변경할 수 없다.
③ final 메서드는 재정의(오버라이딩)할 수 없다.
④ static final 필드는 상수를 말한다.

➢ 예제 4

Parent 클래스를 상속해서 Child 클래스를 다음과 같이 작성했는데, Child 생성자에서 컴파일 에러가 발생했습니다. 그 이유와 해결 방법을 설명해보세요.

```
public class Parent {
    public String name;

    public Parent(String name) {
        this.name = name;
    }
}
```

```
public class Child extends Parent {
    public int studentNo;

    public Child(String name, int studentNo) {
        this.name = name;
        this.studentNo = studentNo;
    }
}
```

➢ 예제 5

Tire 클래스를 상속받아 SnowTire 클래스를 다음과 같이 작성했습니다.SnowTireEx 클래스를 실행했을 때 출력 결과를 작성해보세요.

```
public class Tire {
    public void run() {
        System.out.println("일반 타이어가 굴러갑니다.");
    }
}
public class SnowTire extends Tire {
    @Override
    public void run() {
        System.out.println("스노우 타이어가 굴러갑니다.");
    }
}
public class SnowTireEx {
    public static void main(String[] args) {
        SnowTire snowTire = new SnowTire();
        Tire tire = snowTire;

        snowTire.run();
        tire.run();
    }
}
```